Course Orientation

Dr. Prapong Prechaprapranwong

Computer Engineering

King Mongkut's University of Technology Thonburi

CPE112:
Programming with Data Structures

Learning Outcome:

1. Explain properties and applications of linear data structures
2. Explain properties and applications of non-linear data structures
3. Select and implement appropriate data structures and associated algorithms to efficiently solve programming problems

Programming based on

Try to do your codes better than you have done in CPE100

DRY = Don’t Repeat Yourself

KISS = Keep it Simple, Stupid

https://artandlogic.com/2016/07/four-pounds-nuts-exploring-lean-startup-though-nature-part-ii/young-man-carrying-way-too-much-backpacking-gear-heavy-backpack-shannon-fagan/

https://deviq.com/yagni/

Native Data Structures in Python

There are 4 native data structures in the Python:

- **List** - ordered and changeable

  - Allows duplicate members

- **Tuple** - ordered and unchangeable

  - Allows duplicate members

- **Set** - unordered, unchangeable (remove or add only), and unindexed

  - No duplicate members

- **Dictionary** - ordered (from Python 3.7) and changeable

  - No duplicate members

Abstract Data Type (ADT)

a definition for a data type

- a set of values
- a set of operations allowed on data type

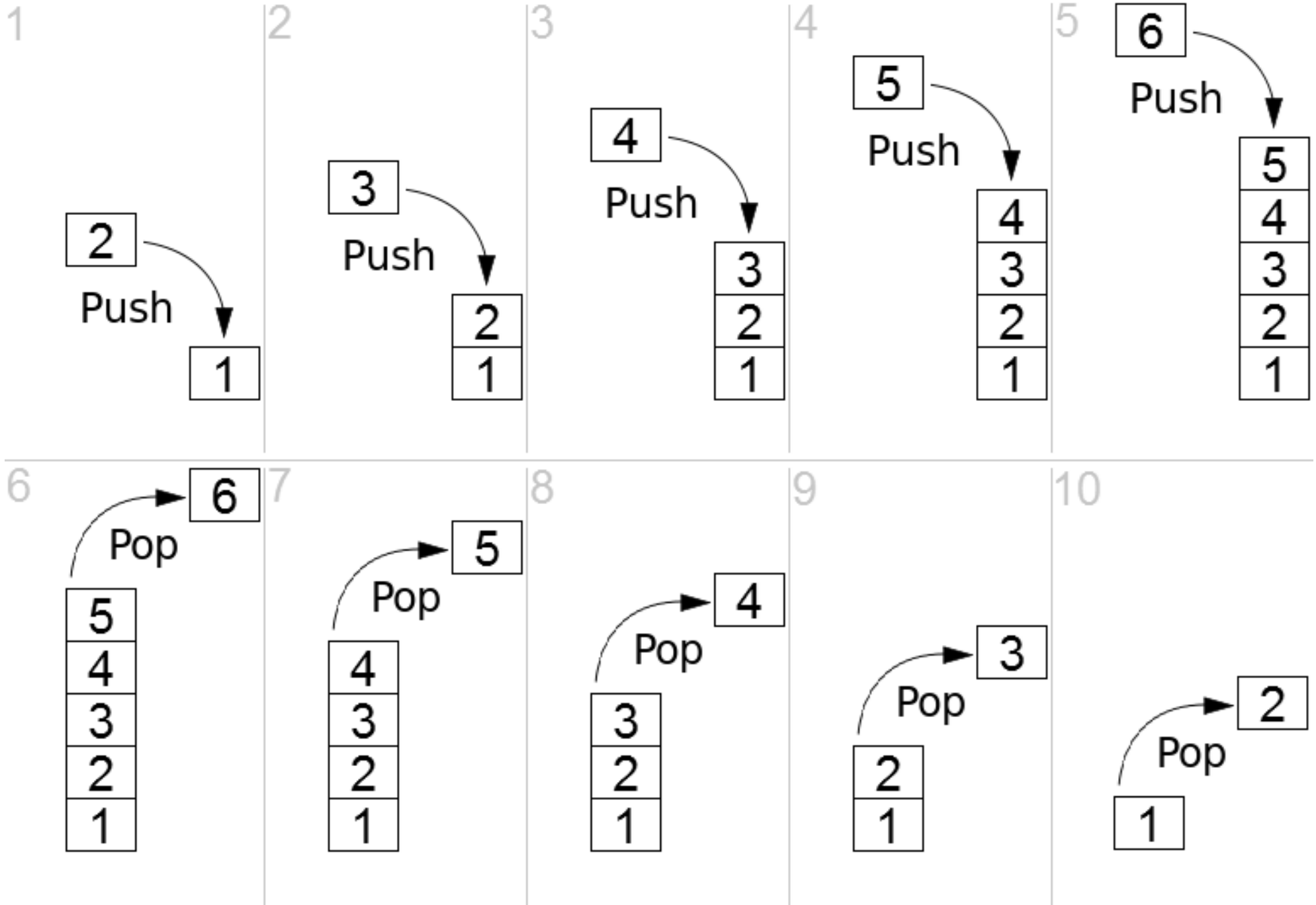
Example of Abstract Data Type :

Stack ADT

an ordered collection of items where items are added to and removed from the end called the “top” (LIFO).

The stack operations are given below.

- **Stack():** creates a new stack that is empty. It needs no parameters and returns an empty stack.
- **push(item):** adds a new item to the top of the stack. It needs the item and returns nothing.
- **pop():** removes the top item from the stack. It needs no parameters and returns the item. The stack is modified.
- **peek():** returns the top item from the stack but does not remove it. It needs no parameters. The stack is not modified.
- **isEmpty():** tests to see whether the stack is empty. It needs no parameters and returns a Boolean value.
- **size():** returns the number of items on the stack. It needs no parameters and returns an integer.


1
2
Push
1
2
3
Push
2
1
3
4
Push
3
2
1
4
5
Push
4
3
2
1
5
6
Push
5
4
3
2
1
6
5
4
3
2
1
Pop
6
7
4
3
2
1
Pop
5
8
3
2
1
Pop
4
9
2
1
Pop
3
10
1
Pop
2

Module

- Module1: Linear Data Structure
- Module2: Non-Linear Data Structure

Tools for Learning and Lab Assignment

- แต่ละ สัปดาห์จะมี Lab assignment ให้ทำเป็นกลุ่ม (3 คน)
- โดยให้เขียนสรุปความเข้าใจที่เรียนเป็นรายงาน บันทึกการทดลอง และตอบคำถามด้วย

https://jupyter.org/

Divide and Conquer

หลักการของ Divide and Conquer

เป็นหลักการแก้ไขปัญหาที่จะลดขนาดของปัญหาลงด้วยการนำปัญหาไปหารออกเป็นส่วนย่อยๆ จากนั้นจึงค่อยแก้ปัญหาที่แบ่งย่อยออกมาและนำกลับมารวมกันเป็นปัญหาเดิม จะมีขั้นตอนในการแก้ปัญหา ดังนี้

1. **Divide** นำปัญหามาหารให้กลายเป็นส่วนย่อยๆ โดยตัวหารส่วนมากแล้วจะมีค่าเป็น 2
2. **Conquer** แก้ไขปัญหาที่ย่อยออกมานั้นทั้งหมด
3. **Combine** นำปัญหาย่อยๆที่แก้ไปแล้วกลับมารวมกันให้หลายเป็นปัญหาเดิม

> Note: เราสามารถนำปัญหามาหารให้กลายเป็นปัญหาย่อยๆกี่ครั้งก็ได้ แต่เราจำเป็นต้องแก้ปัญหาย่อยเหล่านั้นให้หมดและนำกลับมารวมกัน

> Note 2: Divide and Conquer จะต่างจาก Decrease and Conquer ประเภท Decrease by a Constant factor ตรงที่ Decrease by a Constant factor จะลดรูปของปัญหาและนำเพียงปัญหาที่ลดรูปไปแล้วนั้นมาแก้ ส่วน Divide and Conquer จะลดรูปของปัญหาด้วยการแบ่งออกเป็นปัญหาย่อยๆ และแก้ปัญหาที่แบ่งย่อยออกมานั้นทั้งหมด

Algorithm ที่นำหลักการของ Divide and Conquer มาใช้

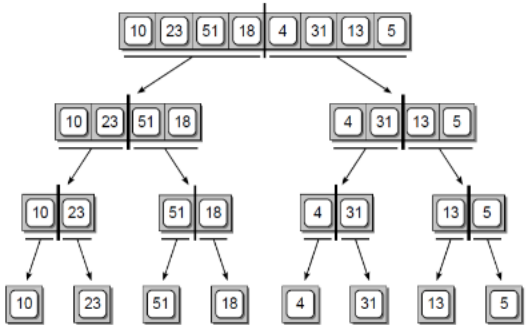
1. Merge Sort
2. Quick Sort
3. Multiplication of Large Integers
4. Strassen's Matrix Multiplication
5. Closet-Pair Problem by Divide and Conquer
6. Convex-Hull Problem by Divide and Conquer

1. Merge Sort

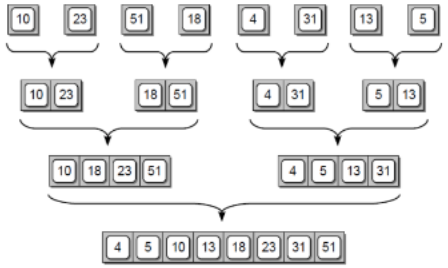
เป็นการจัดเรียงข้อมูลที่นำหลักการของ Divide and Conquer มาใช้ โดยจะแบ่งชุดข้อมูลออกเป็น 2 ส่วนไปเรื่อยๆจนกระทั่งเหลือเพียง 1 ค่าต่อชุดข้อมูล จากนั้นจึงนำชุดข้อมูลที่ถูกแบ่งย่อยไว้กลับมารวมกันจนกลายเป็นชุดข้อมูลที่มีขนาดเท่าเดิม โดยในระหว่างที่นำกลับมารวมกันก็จะมีการจัดเรียงไปด้วย

ขั้นตอนการทำงานของ Merge Sort

1. แบ่งชุดข้อมูลเป็นส่วนย่อยๆ จนเหลือเพียงค่าเดียว



2. นำค่าเหล่านั้นกลับมารวมกันพร้อมจัดเรียงไปด้วย



lecture4.ipynb

```python
def MergeSort(seq):

    # ฟังก์ชั่นสำหรับการนำข้อมูลกลับมารวมกัน
    def Merge(L, R, seq):
        i = j = 0
        while i + j < len(seq):
            if j == len(R) or (i < len(L) and L[i] < R[j]):
                seq[i + j] = L[i]
                i += 1
            else:
                seq[i + j] = R[j]
                j += 1

#-----------------------------------------------เริ่มการทำงานตรงนี้-----------------------------------------------------------------#
    n = len(seq)

    #แบ่งชุดข้อมูลออกเป็นข้อมูลย่อยๆจนกว่าจะเหลือเพียงข้อมูลละ 1 ค่า
    if n > 1:

        # แบ่งชุดข้อมูลออกเป็นฝั่งซ้ายและขวา
        mid = n // 2
        LeftList = seq[:mid]
        RightList = seq[mid:n]
        MergeSort(LeftList)
        MergeSort(RightList)

        # นำข้อมูลกลับมารวมกัน
        seq = Merge(LeftList, RightList, seq)
        return seq
```

[191] Python

```python
Test_list = [4, 1, 10, 8, 7, 12, 9, 2, 15]
MergeSort(Test_list)
print(Test_list)
```

[192] Python

```
[1, 2, 4, 7, 8, 9, 10, 12, 15]
```

ในการทำงานของ Merge Sort จะเห็นว่ามีการแบ่งปัญหาออกเป็น 2 ส่วนไปเรื่อยๆ จากนั้นจึงนำปัญหาย่อยๆเหล่านั้นกลับมารวมกันพร้อมแก้ปัญหาไปด้วย ซึ่งตรงตามหลักการของ Divide and Conquer

2. Quick Sort

เป็นการจัดเรียงข้อมูลที่นำหลักการของ Partition(การแบ่งส่วน) มาใช้ในการแก้ปัญหา โดยจะทำการแบ่งส่วนของข้อมูลไปเรื่อยๆ จนทุกค่าภายในชุดข้อมูลถูกจัดเรียงจนหมด

ตัวอย่างของ Algorithm สำหรับการแบ่งส่วนที่จะนำมาใช้กับ Quick Sort

1. Lomuto Partition
2. Hoare Partition

2.1) Lomuto Partition

เป็น Algorithm สำหรับการแบ่งส่วนที่จะไล่ค่าตั้งแต่ฝั่งซ้ายไปจนถึงฝั่งขวา และเปรียบเทียบค่าเหล่านั้นกับค่าที่เป็นตัวตั้ง(pivot)เพื่อแบ่งส่วน

ขั้นตอนการทำ Lomuto Partition

ตัวแปรที่ต้องกำหนด

- pivot -> เป็นค่าหลักที่จะนำมาใช้ในการเทียบเพื่อแบ่งฝั่ง และจะเป็นค่าที่คั่นระหว่างฝั่งซ้ายและฝั่งขวาในตอนท้าย
- s -> เป็นค่าที่จะระบุตำแหน่งตัวสุดท้ายของค่าในฝั่งซ้าย
- i -> ใช้ในการระบุค่าที่กำลังไล่อยู่ในปัจจุบัน

1. กำหนดค่าในลำดับแรกให้เป็นตัว pivot และให้ตำแหน่ง s อยู่ที่ค่าตัวแรก i อยู่ที่ค่าตัวที่สอง
2. ไล่ค่าจากซ้ายสุดไปยังขวาสุด โดยไล่ไปทีละ 1 ค่า และนำค่าไปเทียบกับตัว pivot โดย
    - หากค่านั้นน้อยกว่าค่า pivot ให้เพิ่ม s ไปหนึ่งตำแหน่ง จากนั้นจึงสลับค่าในตำแหน่งที่ s และ i แล้วเพิ่ม i ไปหนึ่งตำแหน่ง
    - หากค่านั้นมากกว่าหรือเท่ากับค่า pivot ให้เพิ่ม i ไปหนึ่งตำแหน่ง
3. ทำข้อ 2 ซ้ำไปเรื่อยๆจนกว่าค่า i จะไล่ครบทุกค่า จากนั้นให้นำค่าในตำแหน่ง s และค่า pivot มาสลับที่กัน
4. รีเทิร์นตำแหน่ง s ออกมา

Practice 1: โปรแกรม Quick Sort โดยใช้ Lomuto Partition

```python
def QuickSort_Lomuto(seq, l = 0, r = float('inf')):

    def LomutoPartition(A, l, r):
        p = A[l]
        s = l
        for i in range(l + 1, r):
            if A[i] < p:
                s += 1
                A[s], A[i] = A[i], A[s]
```

ให้นักศึกษาแบ่งกลุ่มละ 3 คน และ ลงชื่อใน google form:

https://forms.gle/geAQysVnCTiB4FUb8

ส่ง Assignment บน LEB2

และ ส่งก่อน 23:59

Quiz

ในวิชา มีการ Quiz ทั้งหมด 6 ครั้ง ให้นักศึกษาทำด้วยตนเอง

- ให้นักศึกษาตอบคำถามแบบปรนัย ประมาณ 5-10 ข้อ
- เมื่อทำเสร็จจะทราบคะแนนทันที
- ให้นักศึกษาสามารถ ทำ Quiz ได้ 3 ครั้ง

Examination

ในวิชา มีการสอบ 2 ครั้ง

- ข้อสอบพื้นฐาน – Paper
- ให้นักศึกษาเขียนโปรแกรมตามที่โจทย์กำหนด<u>ด้วยตนเอง</u>
- <u>โดยใช้ module (class) ที่พัฒนาขึ้นเอง</u>
- ให้นักศึกษาทำให้ทันเวลาที่กำหนดแล้ว ส่งทันที
- ในกรณีที่พบว่า มีการลอกโค้ดเพื่อนำมาส่ง จะได้รับ 0 ทั้งผู้ลอกและต้นฉบับ

Reference books

- R0.1 Full speed python
- R0.2 Python Tutorial
- R1 Algorithms in Python
- R2 Data Structures and Algorithms in Python
- R3 Data Structures and Algorithms Using Python
- R4 Open Data Structures (in pseudocodes)

Pretest

- ให้นักศึกษาเขียนโปรแกรมและตอบคำถาม
- เป็นข้อสอบ Pretest ดังนั้น ไม่คิดคะแนนใด ๆ
- ให้นักศึกษาทำเท่าที่ตนเองสามารถทำได้ ข้อไหนทำไม่ได้สามารถข้ามได้